Studio Nx

JS 257 — 조작 연결 (끌기 · 휠 · 키)

2026-10-18

JavaScript 257 — 조작 연결 (`bindControl` · `setControlValue` · `syncAria` · `onControlDown` ~ `onControlKey`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 노브 · 페이더 · 팬을 마우스 · 휠 · 키보드로 다룹니다 (FR-2187)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	256 파라미터 기본값 · 258 칸 안 조작 연결

1. 하는 일

`bindControl` 은 화면 요소 하나에 조작을 묶습니다. 위로 끌면 값이 커지고, Shift 를 함께 누르면 10배 미세 조정이 됩니다. 휠 한 칸은 step 만큼, 화살표 키와 PageUp · PageDown, Home · End 도 값을 바꿉니다.
`setControlValue` 는 새 값을 다듬어 화면 글과 화면 낭독기 값을 함께 맞추고, `commit=true` 이면 서버에 저장합니다. `onControlDown` 부터 다섯 함수는 실제 입력을 받아 그 계산을 부릅니다.

2. 언제 쓰나

쓰는 곳	
`wireControls` (258)	칸 안의 모든 조작 자리에 `bindControl` 을 답니다
`bindModalParam` (261)	상세 모달의 노브에도 같은 조작을 답니다
`wireDemoControls` (262)	예시 화면 노브에 조작만 붙입니다(저장 없음)

3. 코드

```
export function bindControl(el, cfg) {
  if (!el || el.dataset.devBound === '1') return;
  el.dataset.devBound = '1';
  el.devCfg = cfg;
  if (!el.hasAttribute('tabindex')) el.setAttribute('tabindex', '0');
  if (!el.hasAttribute('role')) el.setAttribute('role', 'slider');
  el.addEventListener('pointerdown', onControlDown);
  el.addEventListener('pointermove', onControlMove);
  el.addEventListener('pointerup', onControlUp);
  el.addEventListener('pointercancel', onControlUp);
  el.addEventListener('wheel', onControlWheel, { passive: false });
  el.addEventListener('keydown', onControlKey);
  syncAria(el, cfg);
}

/** 조작 하나의 값을 바꾸고 화면 · aria 를 맞춤 (commit=true 면 서버 저장까지) */
export function setControlValue(el, cfg, v, commit) {
  const item = cfg.item || cfg;
  cfg.value = clampValue(item, v);
```

```
  if (cfg.render) cfg.render(cfg.value);
  syncAria(el, cfg);
  if (commit && cfg.commit) cfg.commit(cfg.value);
}

/** aria-valuenow · aria-valuetext 맞추기 */
function syncAria(el, cfg) {
  el.setAttribute('aria-valuenow', String(cfg.value));
  const text = cfg.type === 'select' ? cfg.value : fmtValue(cfg.item || cfg, cfg.value);
  el.setAttribute('aria-valuetext', String(text));
}

function onControlDown(e) {
  const cfg = e.currentTarget.devCfg;
  if (!cfg || e.button !== 0) return;
  e.preventDefault();
  dragging = {
    el: e.currentTarget,
    startY: e.clientY,
    startNorm: normOf(cfg.item || cfg, cfg.value),
  };
  e.currentTarget.classList.add('is-drag');
  if (e.currentTarget.setPointerCapture) e.currentTarget.setPointerCapture(e.pointerId);
}

function onControlMove(e) {
  if (!dragging || dragging.el !== e.currentTarget) return;
  const cfg = e.currentTarget.devCfg;
  const item = cfg.item || cfg;
  const fine = e.shiftKey ? 0.1 : 1;
  const dn = ((dragging.startY - e.clientY) * fine) / (cfg.pixels || DRAG_PIXELS);
  setControlValue(e.currentTarget, cfg, valueOfNorm(item, dragging.startNorm + dn), false);
}

function onControlUp(e) {
  if (!dragging || dragging.el !== e.currentTarget) return;
  const cfg = e.currentTarget.devCfg;
  e.currentTarget.classList.remove('is-drag');
  dragging = null;
  if (cfg && cfg.commit) cfg.commit(cfg.value);
}

function onControlWheel(e) {
  const cfg = e.currentTarget.devCfg;
  if (!cfg) return;
  const item = cfg.item || cfg;
  e.preventDefault();
  const step = item.step > 0 ? item.step : (item.max - item.min) / 100;
  const dir = e.deltaY < 0 ? 1 : -1;
  const fine = e.shiftKey ? 0.1 : 1;
  setControlValue(e.currentTarget, cfg, cfg.value + dir * step * fine, true);
}

function onControlKey(e) {
  const cfg = e.currentTarget.devCfg;
  if (!cfg) return;
  const item = cfg.item || cfg;
```

```
  const step = item.step > 0 ? item.step : (item.max - item.min) / 100;
  let v = null;
  if (e.key === 'ArrowUp' || e.key === 'ArrowRight') v = cfg.value + step;
  else if (e.key === 'ArrowDown' || e.key === 'ArrowLeft') v = cfg.value - step;
  else if (e.key === 'PageUp') v = cfg.value + step * 10;
  else if (e.key === 'PageDown') v = cfg.value - step * 10;
  else if (e.key === 'Home') v = item.min;
  else if (e.key === 'End') v = item.max;
  if (v === null) return;
  e.preventDefault();
  setControlValue(e.currentTarget, cfg, v, true);
}
```

문법 풀이

식	뜻
`el.dataset.devBound === '1'`	이미 묶인 자리는 두 번 묶지 않습니다
`el.devCfg = cfg`	요소에 조작 설정을 담아 두고 입력 때 꺼내 씁니다
`{ passive: false }`	휠에서 기본 스크롤을 막으려면 필요합니다
`e.currentTarget.devCfg`	지금 다루는 요소의 설정입니다
`e.button !== 0`	마우스 왼쪽 버튼만 받습니다
`setPointerCapture`	끌기가 요소 밖으로 나가도 계속 받습니다
`(startY - clientY)`	위로 끌면 양수가 되어 값이 커집니다
`e.shiftKey ? 0.1 : 1`	Shift 로 미세 조정합니다
끌기는 `commit=false`	다 놓을 때(`onControlUp`) 한 번만 저장합니다
휠 · 키는 `commit=true`	한 번에 한 칸이라 바로 저장해도 요청이 많지 않습니다

4. 왜 이렇게

선택	이유
한 함수로 모든 조작	노브 · 페이더 · 팬이 같은 손맛을 갖습니다
자리 계산으로 처리	값 범위가 파라미터마다 달라도 같은 코드로 다룹니다
끌 때는 저장 안 함	끄는 동안 매번 저장하면 요청이 쏟아집니다
설정을 요소에 담음	함수마다 상태를 따로 넘기지 않아도 됩니다

5. 시험

Bass Synth 카드의 노브를 위로 끌어 `aria-valuenow` 가 `-20` 에서 `0` 으로 바뀌고, 놓는 순간에만 `PATCH` 가 나가는 것을 확인했습니다(끄는 동안 요청 0건, 놓은 뒤 1건). 화살표 키로 한 칸씩, Shift 와 함께 끌면 10배 천천히 움직이는 것도 확인했습니다.